GAMECHANGERS_ANALYSIS

2 5

Game-Changers Analysis — Operator Material (" game changers"), verified + adapted to bash/Go

Field	Value
Revision	1
Created	2026-07-22T21:40:00Z
Last modified	2026-07-22T22:55:00Z
Status	COMPLETE — all 5 game-changers verified against primary sources + literature; 6 pre-flagged concerns resolved with evidence; ADOPT/ADAPT/REJECT verdicts + bash/Go designs for every ADOPT/ADAPT
Author	(T1/main - claude4) BG-GAMECHANGERS deep-research subagent, Fable, §11.4.182
Input	`inputs/OPERATOR_SUPPLIED_GAMECHANGERS_20260722.md` (operator-supplied third-party AI transcript #2, pre-flagged UNVERIFIED)
Sibling inputs	`TOOLING_STACK_VERIFICATION.md` (material #1 verification — the ~2/3-accurate baseline), `EXTERNAL_RESEARCH.md` (Phase 2, R1–R7), `ROOT_CAUSE_ANALYSIS.md`, `solutions/README.md` (SOL-01..10)
Binding constraint	Operator directive 2026-07-22: any codebase we incorporate is **bash + Go (Gin gonic for networking)**. The material's Python/JS code is REFERENCE-ONLY, never an adoption target.
Classification	universal (§11.4.17) — mechanisms carry no project literals except where explicitly marked PROJECT-LAYER (GC-5 flash-seam instantiation); consumers supply paths/thresholds as DATA per §11.4.35
§11.4.224 scope note	This document ships **zero executable artifacts** — every design below is a specification (per §11.4.224(F) prose is not coverage-scoped). Each design names the RED-first test that MUST be written before its implementation exists. No TDD ceremony is faked here.

. Executive verdict table

#	Game-changer	Named-artifact accuracy	Mechanism accuracy	Verdict	Binds at a seam?
GC-1	Mutation testing gate, 100% kill	Tools real (stryker-js, mut-mut, `break` threshold)	**Broken**: hook wiring uses the proven-dead `$CLAUDE_TOOL_INPUT` pattern; 100% whole-corpus kill contradicted by the strongest industrial evidence + formally unachievable (equivalent mutants)	**ADAPT** — diff-scoped Go mutation via `gremlins` + triage-verdict policy, NOT a 100% floor	YES (pre-build/review gate, D1) — but only in our re-wiring; the material's own wiring silently never blocks
GC-2	Adversarial "Saboteur" agent	`.claude/agents/` real; "Code-Hacker" exists but is a competitive-programming arXiv agent, not an installable red-team framework	"Saboteur FAILED if it finds 0 failures" is a §11.4.201(1) FAIL-bluff engine by construction	**ADAPT** — Go native fuzzing (`testing.F`) as the mechanical adversary + saboteur-as-review-role with an evidence-budget verdict, not a must-find-bugs verdict	YES — Go fuzz **corpus replay runs inside plain `go test` forever** (the strongest binding in the whole material, and the material never mentions it)

#	Game-changer	Named-artifact accuracy	Mechanism accuracy	Verdict	Binds at a seam?
GC-3	Cosine-similarity architecture gate	chromadb / sentence-transformers / langchain real; `archgate` REAL but characterized — it is a **deterministic** ADR-rules CLI, not a vector gate	Embedding-similarity as a *blocking* gate is a false-positive generator with uncalibrated, mutually-inconsistent thresholds; non-deterministic across model versions (§11.4.50 violation)	**REJECT mechanism / ADAPT goal** — deterministic import-graph conformance via `go-arch-lint` + a structure-matched bash scanner	YES in the deterministic replacement form (exit-code gate at pre-build); NO in the embedding form (a gate nobody can trust binds nothing)
GC-4	50-line atomic delivery, ">90% reduction"	No tool claims to verify	"50 lines" is an invented law — real literature says 100–400 LOC at the **review** seam, with explicit carve-outs; ">90% in internal trials" has **no located citation** — treated as fabricated-or-unverifiable	**ADAPT as advisory review-size discipline / REJECT the hard session-halting gate**	NO as a hard gate (honest binding impossible without §11.4.201(1) false refusals); advisory annotation at the review-dispatch seam only

#	Game-changer	Named-artifact accuracy	Mechanism accuracy	Verdict	Binds at a seam?
GC-5	Live telemetry shadowing	Istio `mirror` / `mirrorPercentage` real (`mirrorPercent` = older field name)	Mechanism REAL for HTTP services; **does not transfer to firmware** — there is no production request stream to mirror	**REJECT mechanism / ADAPT principle** — canary-ordered flash promotion gate + recorded-input-trace replay; A/B auto-rollback REJECTED for this target (frozen A13 U-Boot, §11.4.133) with a §11.4.112(5)-bounded scope statement	YES — a flash-seam promotion gate (device-2 flash refused until device-1 burn-in verdict exists + green)
—	Unified pipeline: "the system literally cannot produce a false green light"	`zero-bluff-gates` repo = self-acknowledged placeholder	**REJECT as stated** — no detector composition yields "cannot"; residual classes remain (equivalent mutants, oracle weakness, the coverage blind complement per `solutions/README.md` §1)	Honest framing: each layer shrinks a *named* bluff class; none eliminates all of them	—

Gin gonic usage: ZERO services proposed. Per the operator's own constraint ("Gin only where networking is genuinely needed — do not invent a server"): none of the five adapted mechanisms needs a network service. The only networking candidate in the material (GC-5's `get_shadow_report` MCP server) is rejected together with its mechanism — building a Gin server to serve reports about a shadow environment we don't have would be infrastructure theater.

Accuracy vs material #1 (~2/3 accurate, ~1/3 confabulated): material #2 scores **better on artifact existence** (9 of 11 named artifacts exist — see §8 tally) but **the same or worse on load-bearing mechanism** — the glue that would make anything actually fire (the hook wiring, the 100% threshold, the >90% effect size, the "cannot produce a false green" guarantee) is where it confabulates, and those are precisely the parts an adopter would trust without testing. Net: **≈70% named-artifact accuracy, ≈40% mechanism accuracy.** The pattern from material #1 repeats exactly: real tools, fabricated integration.
[1]

11 4 201 7

. Method + instruments (§ . . ( ) compliance)

- **Web verification**: 10 web searches + 4 targeted page fetches, all access-dated **2026-07-22** (footer §10). Every existence claim below carries its check.
- **Control needles for absence claims**: every "not found" verdict in this document was reached through a search path that returned rich results for a sibling known-present query in the same round (e.g. the round that failed to find any citation for ">90% reduction in internal trials" successfully returned the SmartBear/Cisco study, the Google small-CL guidance, and Purushothaman & Perry through the same instrument) — the instrument was proven able to see before any zero was read as absence. `find -newermt` was not used anywhere (known-unreliable on this host).
- **Local verification**: the `$CLAUDE_TOOL_INPUT` verdict is NOT re-derived here — it was already proven with captured doc-fetch evidence in `TOOLING_STACK_VERIFICATION.md` (row: "Force-push-block example using `$CLAUDE_TOOL_INPUT` … BROKEN AND DANGEROUS AS A PATTERN", docs

fetch 2026-07-22, env-var roster `CLAUDE_PROJECT_DIR` / `CLAUDE_PLUGIN_ROOT` /… contains no `CLAUDE_TOOL_INPUT`). This document cites that captured evidence rather than re-fetching.

- **Honest boundary (§11.4.6)**: "no citation found" ≠ "no citation exists". Where this document says a quantitative claim is unverifiable, that is the earned claim; fabrication is asserted only where the claim shape (a precise effect size with no source, from a source with a proven fabrication record) makes unverifiable-as-published the operative fact either way.

---

2 1 . GC- — Mutation Testing Gate ("kill fake tests")

. Claim-by-claim verification

Claim	Verdict	Evidence (access 2026-07-22)
`stryker-js` exists, JS/TS mutation testing	**REAL**	stryker-mutator.io docs fetched
`thresholds { break: 100 }` config exists and fails the build below the score	**REAL** — `break` default is `null`; `mutation score < break → exit 1`	StrykerJS configuration docs fetched: "mutation score < break: Error! Stryker will exit with exit code 1"
`mutmut` exists, Python mutation testing	**REAL, ACTIVE** — v3.3.1 (Nov 2025), boxed/mutmut	web search + readthedocs
PreToolUse hook running stryker on `git commit`, via `$CLAUDE_TOOL_INPUT`	**BROKEN — the identical fabricated pattern from material #1.** No `CLAUDE_TOOL_INPUT` env var exists in the documented hook environment; the variable expands empty, the match never fires, the hook **silently never blocks** — the §11.4.201 false-negative shape, again. Concern 1 CONFIRMED.	`TOOLING_STACK_VERIFICATION.md` (captured docs-fetch evidence, 2026-07-22); real hooks read stdin JSON, as our §11.4.109 guards do
`fix_mutants.sh` loop feeding survivors to `claude --print`	Plausible to build (headless `claude -p` is real per §11.4.187) but **no such published artifact located**; treat as a sketch, not a citation	control-needled search round

Claim	Verdict	Evidence (access 2026-07-22)
"Hard rule: ZERO surviving mutants; feature not complete until 100% kill"	**CONTRADICTED — see §2.2**	TSE'22 + equivalent-mutant problem

. Concern resolved: is % whole-corpus kill achievable, or a gaming-inducing target?

It is unachievable in general, and the strongest industrial evidence says even attempting the whole-corpus form collapses. Three independent lines:

1. **Google (the largest running mutation deployment, ~24k developers): diff-scoped at review, never whole-corpus.** Petrović, Ivanković, Fraser & Just — *Practical Mutation Testing at Scale* (IEEE TSE 2022) + ICSE 2021 (already captured in `EXTERNAL_RESEARCH.md` R1.3): whole-codebase mutation "does not scale and is not necessary"; mutation effort belongs at the **changed lines**; unfiltered mutants produce "mutant fatigue" and get ignored. Google does not enforce a kill *score* at all — it surfaces *filtered, relevant* surviving mutants as review findings.
2. **Equivalent mutants make a literal 100% floor formally unsatisfiable.** Some mutants are semantically identical to the original program (equivalence is undecidable in general); a surviving equivalent mutant can never be killed by any test. A mechanical `break: 100` therefore terminates in one of exactly three states: (a) permanent block, (b) a human exemption channel (at which point the rule is no longer "100%, zero exceptions" — it is a triage policy, which is what we design below), or (c) **gaming** — tests written to memorize mutant-killing outputs rather than assert behaviour, the precise §11.4 metric-layer PASS-bluff that §11.4.224(C) just fenced for coverage. Tooling agrees: gremlins itself reports `LIVED` / `NOT VIABLE` as expected, normal outcomes.
3. **Where the real leverage is:** OOPSLA 2025 (R2.5) — one property-based test kills ~50× the mutants of one unit test. If the goal is kill-rate, the efficient lever is **stronger oracles** (properties, fuzz targets — see GC-2), not a numeric kill floor.

Direct consequence for us: we just landed §11.4.224 with a ≥85% coverage floor whose gameability we had to fence at the contract layer (RED-capable-tests-only numerator, exclusion-list fence). A 100% mutation-kill floor is the same shape one level up, with a harder gaming incentive and a formally unsatisfiable target. **The material's hard rule is rejected; the mechanism is adapted as a triage-verdict gate (D1, §7.1).**

. Concern resolved (mutation half): is automated mutation an upgrade over § . hand-authored paired mutations?

Complement, not replacement — they catch disjoint classes:

- **§1.1 paired mutations** are *semantic*: hand-authored to strip the exact invariant a gate claims to enforce ("the mutation IS the test-first artifact for a gate", §11.4.224(A)). No automated operator set produces "remove the control-needle check" or "flip the verdict-file polarity". Automated mutation CANNOT replace them.
- **Automated diff-scoped mutation** is *broad and shallow*: arithmetic/relational/boolean operator flips over every changed line, catching the assertion-free-test and weak-oracle classes at scale — exactly what hand-authoring cannot afford to cover across a whole diff. §1.1 CANNOT replace it either.
- **The genuine upgrade claim is therefore TRUE but only for our Go surfaces** (workable-items engine, continuum, docs_chain, report engines — all "smallish modules", exactly gremlins' documented sweet spot), and only diff-scoped. For **bash** there is **no located mutation-testing tool of any maturity** (control-needled search round: the same instrument that returned gremlins/mutmut/stryker returned nothing bash-specific; `UNKNOWN:` whether an obscure one exists) — for bash, §1.1 hand-authored mutations REMAIN the only mechanism, now backed by §11.4.224(E)'s PS4-trace coverage floor.

. Go tooling verification (the operator-mandated ecosystem check)

Tool	Status (2026-07-22)	Fit
`go-gremlins/gremlins`	**REAL, ACTIVE** — v0.6.0 (2025-12-06), 374 stars, issues active into Jan 2026; `gremlins unleash` ; reports RUNNABLE/NOT COVERED/KILLED/LIVED/TIMED OUT/ NOT VIABLE; documented limitation: "doesn't work very well on very big Go modules — a run can take hours"; 0.x — config may change between minors	**SELECTED** — our Go modules are exactly its documented sweet spot; run per-package on changed packages only, with timeout
`avito-tech/go-mutesting` (fork of zimmski/)	REAL but **dormant** (inactive since late 2025; upstream zimmski has an "is go-mutesting dead?" issue #100)	Rejected — maintenance risk
`zimmski/go-mutesting`	Effectively unmaintained	Rejected

. Verdict: ADAPT → Design D (§ .)

Diff-scoped, triage-verdict, Go-only, wired as a real gate (`CM-*` class) — never the material's silent-no-op hook, never a 100% floor.

---

3 1. GC- — Adversarial "Saboteur" agent

. Claim-by-claim verification

Claim	Verdict	Evidence
`.claude/agents/<Name>.md` custom-agent files	**REAL** (established Claude Code feature; already referenced with source evidence in `TOOLING_STACK_VERIFICATION.md` §4)	prior captured evidence
"CodeHacker framework"	**NAME REAL, CHARACTERIZATION STRETCHED** — arXiv 2602.20213: an LLM-agent generating adversarial test cases for **competitive-programming submissions** (WA/RE/TLE/MLE verdicts). It is a paper, not an installable general red-team framework for production services. Citing it as the pedigree for a production saboteur is the carrier-vs-thing pattern (§11.4.201(7)(a)).	arXiv + HF paper page, 2026-07-22
`adversarial_runner.py` fuzz harness	No published artifact located (control-needled); reference sketch only	search round
"Gate: 100+ disruptive cases with no crash"	Reasonable *budget* shape — converges with §11.4.85's closed-set stress floor (N ≥ 100 iterations) which we already mandate	§11.4.85
"The Saboteur has FAILED if it finds 0 failures ('try harder')"	**REJECT as a hard rule** — on genuinely robust code this mandates either an infinite loop or a fabricated finding; it is a §11.4.201(1) FAIL-bluff engine by construction (a false-positive refusal machine). The honest form is an **evidence budget**: N cases across the closed chaos classes, executed and captured, verdict from evidence — which is exactly §11.4.85 + §11.4.118 discovery-pressure + SOL-09 detection-pressure. Zero findings after a proven-executed budget is a legitimate PASS with enumerated coverage (§11.4.118: "here is the enumerated set we exercised").	§11.4.201(1), §11.4.85, §11.4.118

. Concern resolved (adversarial half): does it BIND where § . . prescribes?

This is the decisive finding of the whole analysis. §11.4.85 prescribes stress+chaos in prose + per-fix suites; BG-TOOLING-VERIFY found our only two unoccupied seams are **PostToolUse detective hooks** and the **Stop-seam completion gate**. The material's saboteur, as written (an agent role invoked "before done"), is *prose-adjacent* — nothing forces it to run. Two mechanisms make adversarial pressure actually bind:

1. **Go native fuzzing's corpus replay — the strongest binding available, and the material never mentions it.** Go fuzzing (`testing.F` , coverage-guided, first-class since Go 1.18 — verified via go.dev/doc/security/fuzz, 2026-07-22) has the property that **every crasher found is minimized into `testdata/fuzz/<FuzzTarget>/` and replays as a plain test case in every ordinary `go test` run forever** — no flags, no fuzz mode, no orchestration. A found failure becomes a permanent §11.4.135-class regression guard *mechanically, at zero marginal machinery*. This is a seam-binding our hand-authored §11.4.85 suites do not have. The SQLite fact already in our research (R4: **22 AFL-findable crashers at 100% MC/DC coverage**) is the quantitative proof fuzzing finds what coverage-complete suites structurally cannot.
2. **Verdict-file consumption at the Stop/release seam.** A saboteur/fuzz run that writes a machine-readable verdict (cases run, classes exercised, findings, evidence paths) into the SOL-01/SOL-02 verdict store gives the Stop-seam completion gate (the fablize-referenced design in `TOOLING_STACK_VERIFICATION.md` §6) something to refuse on. An agent whose output is prose binds nothing; an agent whose output is a verdict file consumed by a refusing seam binds.

. Verdict: ADAPT → Designs D (Go fuzz work-class + bounded fuzz gate, § .) and D b (saboteur-as-review-role with evidence-budget verdict, § .)

---

4 1. GC- — Semantic contract checking via vector RAG ("Achilles heel scan")

. Claim-by-claim verification

Claim	Verdict	Evidence
`chromadb`, `sentence-trans-formers` (`all-MiniLM-L6-v2`), `lang-chain`	**REAL** — chromadb defaults to all-MiniLM-L6-v2 (384-dim) via sentence-transformers	web search 2026-07-22
"block if cosine similarity > 0.85 (elsewhere: distance < 0.3)"	**INTERNALLY INCONSISTENT** — for cosine metrics, similarity 0.85 ≡ distance 0.15, not 0.3; the two statements of the same threshold disagree by 2×. No calibration procedure, no corpus, no FP/FN characterization given anywhere. Magic numbers presented as law.	arithmetic; input doc lines 32
`archgate` "(dotnet tool)"	**REAL TOOL, DOUBLY MISCHARACTERIZED** — archgate.dev / github.com/archgate/cli exists (v0.50.0, 2026-07-17, 55 stars, Apache-2.0, OpenSSF badge): it enforces ADRs as **deterministic executable rules** (`.rules.ts` companion files reporting file+line+violated-ADR), implemented in TypeScript, installable via npm/pip/dotnet/go/gem among others. So: (a) "dotnet tool" is at best one of eight install channels, not what it is; (b) far more damning, **the material cites a deterministic-rules tool as support for an embedding-similarity mechanism the tool does not use** — the material's own best citation refutes its own mechanism.	GitHub fetch 2026-07-22

. Concern resolved: can a cosine-similarity blocking gate be made trustworthy?

No — for reasons that are structural, not tuning problems:

1. **False-positive economics.** Our Phase 2 evidence: 90% override rates on mis-calibrated alert fleets (R6.3); Google's operational doctrine that review-blocking checks must stay under ~10% *effective* false positives or developers delete the check's authority (CACM 2018, R5.2). An uncalibrated similarity threshold over embeddings of code-vs-ADR-prose starts far above that bar. A gate that misfires spends its trust budget and then binds nothing — worse than absent, because it teaches bypass.
2. **Similarity is not entailment.** High cosine similarity between new code and an anti-pattern document proves topical proximity, not violation — the code *fixing* an anti-pattern embeds close to the anti-pattern's description. Both error directions are intrinsic, not tunable away.
3. **Non-determinism (§11.4.50).** The verdict changes when the embedding model changes (model updates, quantization, hardware). A gate whose PASS/FAIL flips on identical input across runs violates deterministic-consistency and can never carry a §1.1 mutation pair honestly.
4. **§11.4.201(1)**: a false-positive refusal is a FAIL-bluff. A mechanism whose false positives are structural is a FAIL-bluff *engine*.

Where the retrieval idea IS legitimate: as *advisory context injection* — retrieving relevant ADRs into the review/authoring context (what archgate's "briefings" actually do, and what our CodeGraph mandates §11.4.78/79 already cover). Advisory retrieval + deterministic gate is the honest split.

. The honest replacement — deterministic architectural conformance (verified)

Tool	Status (2026-07-22)	Notes
`fe3dback/go-arch-lint`	**REAL, ACTIVE** — YAML component/dependency rules over the Go import graph; exit 0 clean / 1 on violation; Docker image; CI-ready; hexagonal/onion/DDD patterns	**SELECTED** for Go
ArchUnit / ArchUnit-NET	REAL (Java/.NET lineage — prior art for the pattern)	Not our stack; cited as pedigree
dependency-cruiser	REAL (JS import rules — same pattern)	Not our stack
archgate/cli [4] [4] [3]	REAL, deterministic ADR rules, but TypeScript rule files — adopting it would put our rule logic in TS against the bash/Go constraint [7] [3]	Assessed as prior art; concept (ADR → executable rule) is already our CM-gate + §1.1 pattern

. Verdict: REJECT mechanism / ADAPT goal → Design D (§ .)

What D3 still does NOT catch (honest boundary): semantic contract violations not expressible as import/dependency/structure rules — wrong locking order, misused API sequencing, violated timing contracts. Those remain the §11.4.194 exhaustive-review's job and partially GC-2's fuzz targets. No mechanical gate closes that class; claiming otherwise would be the material's own overreach.

. GC- — Progressive Atomic Delivery ("Zeno compiler", -line units)

. Concern resolved: what does the change-size literature actually support?

Verified findings (all access 2026-07-22):

1. **SmartBear/Cisco (2,500 reviews, 3.2 MLOC, 50 developers)** — the real numbers are **200–400 LOC per review** for 70–90% defect discovery, with detection collapsing beyond ~400 LOC and above ~450–500 LOC/hour review speed. This is a **review-batch-size** finding, not an implementation-unit-size law. Known caveats we must carry: observational study by a review-tool vendor; the inverse defect-density-vs-size curve is partly a denominator artifact (defects/kLOC mechanically falls as kLOC grows); 61% of reviews found zero defects.
2. **Google eng-practices (small CLs)** — "100 lines is usually a reasonable size for a CL, and 1000 lines is usually too large"; file count matters; **explicit carve-outs where large is fine**: deletions, trusted-tool automated refactors. Guidance, explicitly not a hard gate.
3. **Purushothaman & Perry (IEEE TSE 2005)** — even **one-line changes carry ~4% error probability**; ~10% of maintenance changes are one-liners. Small ≠ safe; the risk floor never reaches zero, which alone falsifies "atomic units ⇒ cuts false success by >90%" as a mechanism (the units still fail at a base rate).
4. **The "50-line" number**: **no located source** (control-needled — the same searches returned the three studies above). It sits 4–8× below the evidenced review-effectiveness band. Invented precision.
5. **">90% reduction in false success in internal trials"**: **no citation given, none located.** From a source with a proven fabrication record, an exact effect size with no methodology is treated as unverifiable-marketing; it is not evidence.

. Where hard small-diff mandates actively harm (the operator asked for this explicitly)

- **Atomicity inversion:** a coherent cross-cutting change (schema + all call sites; an interface change + implementors) split to fit a 50-line cap produces intermediate states that do not compile or do not pass — directly contradicting

the material's *own* "each unit must compile in isolation and pass tests in a clean sandbox". The two rules in GC-4 are mutually inconsistent for exactly the changes that matter most.

- **Context fragmentation at review:** reviewers of fragment N lack the design context of fragments N+1..M (Google's own "the reviewer often has no context" cuts both ways); semantically-coupled fragments reviewed in isolation hide interaction defects — the §11.4.194 multi-factor class.
- **Gaming channel:** a hard cap incentivizes reclassifying code as "generated", splitting tests from the code they test, or landing scaffolding-then-behaviour sequences where every fragment is individually plausible and the composite is never reviewed as a whole.
- **Project reality collision:** §11.4.9 deliberately BATCHES source fixes before rebuild because rebuild latency (5–7 h containerized AOSP) dominates; a session-halting per-50-lines gate would serialize operator time onto rebuild latency — the exact anti-pattern §11.4.9 exists to prevent. And "block commits >5 files / >50 lines per file" would have blocked essentially every legitimate landing in this repository's history, including the governance commits that created the anti-bluff regime itself.
- **The "one real CRUD against a live staging DB via MCP" clause** transfers only to service backends; for firmware work-classes the analogous obligation already exists as §11.4.108's runtime-signature-on-clean-target — stricter than the material's version.[5][3][4][7][4]

. Verdict: ADAPT advisory / REJECT hard gate → Design D (§ .)

The evidenced discipline (keep *review units* in the 100–400 LOC band; flag oversized diffs for extra review depth; never block) lands as an advisory annotator in the §11.4.125/142 review dispatch. A hard gate at 50 lines would be a §11.4.201(1) false-refusal machine with a gaming channel, and is rejected with the same reasoning §11.4.224(E) used to fence the coverage floor.

. GC- — Live telemetry shadowing → the firmware question

. Claim verification

Claim	Verdict	Evidence
Istio `mirror` / `mirrorPercent` traffic mirroring	**REAL** — VirtualService `mirror:` + `mirrorPercentage.value` (the `mirrorPercent` spelling is the older field generation); fire-and-forget shadow copies, `-shadow` Host suffix	istio.io mirroring task, 2026-07-22
k8s shadow deployment, 10k requests / 24 h / <0.1% deviation gate	Mechanism real for HTTP services; the specific numbers are unsourced policy choices	—
MCP server exposing `get_shadow_report` / `get_shadow_failures`	Buildable, no published artifact located; moot without a shadow environment	—

. Concern resolved: does shadow validation transfer to firmware at all?

The mechanism does not transfer. A bounded piece of the principle does. We ship firmware to physical devices; there is no production HTTP request stream to mirror, no request/response pair to diff, and no shadow environment that is not simply *another physical device*. Forcing the Istio pattern onto this topology would produce infrastructure with no input. That is the honest "this one does not transfer" the dispatch asked for — for the *mechanism*.

What the embedded industry actually does at this seam (verified practice, Memfault OTA guides + industry write-ups, 2026-07-22): canary device cohorts (≈1% first, hardware-variant-diverse) → staged ring expansion with per-ring time+metric gates (24–72 h) → health telemetry (crash rate, connectivity, power)

with halt-on-degradation → A/B slot bootloaders with confirm-or-rollback watchdogs. This is the firmware analogue of shadow validation, and it is real, mature practice — not a fantasy.

Mapping to our topology, honestly, piece by piece:

Embedded practice	Transfer verdict for this project
Canary cohort → staged rings	**TRANSFERS, mostly exists already** — a 2-device fleet's canary structure is "flash D1, burn-in, then D2", which §11.4.130/§11.4.132 already order in prose. What is missing is the SEAM: nothing mechanically refuses the second flash while the first device's burn-in verdict is absent or red. Design D5 closes that.
Health-metric gates during burn-in	**TRANSFERS** — §11.4.128 always-on recording + Arvus sink probes ARE the telemetry source; D5 consumes a burn-in verdict computed from them (crash/ANR census, boot loops, audio XRUN census — signals we already capture).
A/B slots + bootloader auto-rollback	**REJECTED FOR THIS TARGET** — requires bootloader participation; this project's U-Boot is a frozen A13 binary whose rebuild is a **proven brick** (project CLAUDE.md: patched A15 U-Boot MD5 `1ca759…` = bricks device; MD5-pinned loader), and §11.4.133 blocks unverifiable bootloader surgery outright. **§11.4.112(5)-bounded scope statement:** infeasible-without-accepted-brick-risk *on Orange Pi 5 Max with the frozen A13 U-Boot*; NOT claimed impossible in general — adjacent goals not covered by this verdict: (a) the Forlinx production board (T4 domain) may ship proper A/B bootctrl and MUST be evaluated on its own evidence; (b) AOSP seamless A/B (`update_engine`) is standard on targets whose bootloaders support slots. Neither neighbour inherits this rejection.
Replayed traces (the closest true analogue of "mirror real traffic")	**TRANSFERS GENUINELY** — we hold §11.4.128 recorded corpora including real input sequences; replaying a recorded real-user input trace against the NEW build on the canary device, diffing the health census against the same trace's census on the previous build, is input-replay shadow validation. Composes §11.4.199 (exact-sequence) and §11.4.143 (real-user-journey). Folded into D5 as the burn-in content.
HIL rigs	**ALREADY EXIST HERE** — the physical D1/D2 devices + HDMI/Arvus sink probes are the HIL rig; nothing to buy or build.

. Verdict: REJECT mechanism / ADAPT principle → Design D (§ .) — PROJECT-LAYER instantiation (flash seam is project-specific per § . . ; the promotion-gate pattern itself is universal).

. Designs (bash + Go only; zero Gin services — none needed)

Every design below is a SPECIFICATION. Per §11.4.224(A), when implemented, **the named RED test is written and observed to fail FIRST**; per §1.1 each gate ships its paired mutation. Feed-forward: these are candidate SOL-11..15 entries for the Phase 3 solutions agent (`solutions/README.md` currently ends at SOL-10; the Stop-seam and PostToolUse designs from `TOOLING_STACK_VERIFICATION.md` §6 are the consumers of D1/D2's verdict files — cross-reference, do not duplicate).

. D — Diff-scoped Go mutation gate (`mutation_gate.sh` + gremlins)

- **Seam:** pre-build / review (runs in the §11.4.125 review window; verdict file consumable by the Stop-seam gate).
- **Mechanism (bash orchestrator, ~120 lines):**
  1. `base=$(git merge-base HEAD origin/main)` ; changed Go packages = `git diff --name-only "$base"..HEAD -- '*.go'` mapped through `go list` .
  2. Per changed package: `gremlins unleash --tags=... <pkg>` under `timeout` (per-package budget, consumer DATA), output parsed to per-mutant records `{file, line, operator, status}` .
  3. **Scope filter:** only mutants whose `file:line` intersects the diff's changed lines count (the TSE'22 diff-incremental discipline — whole-corpus results are reported but never gated on).

4. **Policy (NOT a score floor):** every in-diff `LIVED` mutant requires a recorded triage verdict in a checked-in ledger: `killed-by-new-test | equivalent (justification) | accepted-risk (tracked §11.4.197 item id)`. Unlisted survivor ⇒ gate FAILs naming file:line:operator. This is the §11.4.135/§11.4.224(E) checked-in-exemption pattern instead of the unachievable `break: 100`.
5. Verdict file (JSON: mutants generated/killed/lived/triaged + evidence path) written to the SOL-02-class verdict store.

- **RED-first test:** a fixture Go package with a deliberately assertion-free test + a live in-diff mutant; the gate MUST FAIL on it before any GREEN run is trusted. Golden-good: same package with a killing test ⇒ PASS. Negative control (§11.4.201): an out-of-diff `LIVED` mutant MUST NOT fail the gate.
- **Paired §1.1 mutation:** strip the scope filter (so the gate silently ignores in-diff survivors) ⇒ RED fixture must flip to PASS ⇒ meta-test FAILs.
- **Does NOT catch:** equivalent-mutant misjudgment by the triager; bash code (no bash mutation tool exists — §2.3, stays on hand-authored §1.1); semantic invariants outside gremlins' operator set; test quality beyond mutant-killing (oracle relevance stays §11.4.194's job).

. D — Go fuzz work-class + bounded fuzz gate (`fuzz_gate.sh`); D b saboteur-as-review-role

- **D2 seam:** pre-build gate + permanent regression binding via corpus replay.
- **Mechanism:**

1. **Work-class rule (universal, DATA per §11.4.35):** every Go package that parses external input or implements a state machine (parsers, verdict stores, claim registries, sync engines) MUST carry ≥1 `FuzzXxx(f *testing.F)` target with seeded corpus in `testdata/fuzz/` **committed** — because committed corpus entries replay in every plain `go test` run, every crasher ever found becomes a permanent §11.4.135-class guard with zero orchestration.
2. `fuzz_gate.sh` (bash, ~80 lines): enumerates fuzz targets (`go test -list '^Fuzz'` per package — a real execution, not a grep); asserts declared work-class packages have ≥1 target; runs each with

`-fuzz=<target> -fuzztime=<budget>` (consumer DATA, e.g. 60s/target) in the heavy diff-gated tier per §11.4.110(4) two-speed honesty; plain corpus replay runs in the always-on tier.

3. New crashers: minimized input committed + tracked item minted via §11.4.202/§11.4.214 intake (dedup-linked, never a new id for a recurring crasher).

- **RED-first test:** fixture package with a known-crashing seed (e.g. an off-by-one slice parser) — gate MUST FAIL/report the crasher before trust; golden-good sibling passes.
- **Paired §1.1 mutation:** delete the corpus-replay assertion ⇒ crasher fixture goes green ⇒ meta-test FAILs.
- **D2b (saboteur role):** a `.claude/agents/saboteur.md` review-time role whose toolbelt is D2's fuzz targets + §11.4.85 `stress_chaos.sh` helpers, and whose REQUIRED output is a machine verdict file `{cases_run, classes_exercised[], findings[], evidence_paths[]}` in the verdict store — consumed by the Stop-seam completion gate (TOOLING_STACK_VERIFICATION §6 design). **Success semantics: evidence budget met + all findings triaged — NEVER "must find bugs"** (§3.1). Without the verdict-file consumption, D2b is prose and binds nothing; with it, "done" mechanically requires the adversarial pass to have RUN.
- **Does NOT catch:** logic errors that never crash/panic (fuzzing needs an oracle — pair with property assertions inside targets where invariants exist, per the [7] [3] [3] OOPSLA'25 PBT evidence); non-Go surfaces; concurrency bugs outside the fuzzed entry points (`go test -race` stays a separate obligation).

. D — Deterministic architecture gate (`arch_gate.sh` + go-arch-lint)

- **Seam:** pre-build, always-on tier (fast).
- **Mechanism:**

1. Consumer-DATA `.go-arch-lint.yml` per Go module (components + allowed dependency edges + vendor allowlists — the engine/consumer-data split the constitution already knows: e.g. "reporting engine may not import project-literal packages" becomes a checkable edge rule supporting §11.4.28 decoupling).

2. `arch_gate.sh` : runs `go-arch-lint check --project-path <module>` per Go module (exit 1 = violation, file+import named); for bash, a structure-matched forbidden-pattern scanner (word-boundary/AST-adjacent matching per §11.4.201(7)(a), each scan control-needled with a same-dialect known-present needle per §11.4.201(7)(b)) for consumer-declared forbidden calls (e.g. raw `git push` outside wrappers — the §11.4.113 class).
3. ADR linkage without embeddings: each rule row carries the anchor/ADR id it enforces — the archgate *concept* (decision → executable rule) implemented in our existing CM-gate idiom.

- **RED-first test:** fixture module with one forbidden import ⇒ gate FAILs naming it; golden-good sibling passes; negative control: a *comment mentioning* the forbidden import MUST NOT fire (the carrier case).
- **Paired §1.1 mutation:** delete the rule row ⇒ RED fixture passes ⇒ meta-test FAILs.
- **Advisory retrieval half (non-gating):** relevant-ADR context injection at authoring time stays in CodeGraph/§11.4.78-79 territory — explicitly NOT a blocking mechanism.
- **Does NOT catch:** contracts not expressible as import/structure rules (locking order, call sequencing, timing) — §11.4.194 review + D2 fuzz targets own those; violations *inside* an allowed edge.

. D — Review-size annotator (advisory, never blocking)

- **Seam:** §11.4.125/§11.4.142 review dispatch.
- **Mechanism (bash, ~40 lines):** compute `git diff --stat` vs merge-base; annotate the review dispatch with total churn, per-file churn, and a flag when any reviewed unit exceeds the evidence band (>400 changed LOC per SmartBear/Cisco; >~1000 total per Google) ⇒ instruction to the reviewer: split the REVIEW into passes ≤400 LOC and state per-pass coverage — never halt the work. Carve-outs mirrored from the evidence: deletions and generated/vendored churn (identified via the §11.4.224(E) exclusion-list classes) excluded from the count.

- **RED-first test:** fixture diff of 1200 lines ⇒ annotator emits the flag; 80-line fixture ⇒ no flag; deletion-only 5000-line fixture ⇒ no flag (negative control — the §11.4.201(1) false-refusal guard).
- **Explicitly rejected:** any blocking form; the 50-line cap; the >5-files cap (§5.2 evidence).
- **Does NOT catch:** anything by itself — it is a review-quality aid; defect detection stays with the review + gates it feeds.

. D — Canary promotion gate at the flash seam (`canary_promote_gate.sh`) — PROJECT-LAYER

- **Seam:** `scripts/flash.sh` wrapper (the only sanctioned flash path).
- **Mechanism (bash, ~100 lines):**
  1. Consumer DATA: ordered device roster `[canary=D1(998fd36615e99484), fleet=D2(66ff9c4f51f00ee7)]` + burn-in budget (e.g. ≥N hours or the §11.4.130/§11.4.132 targeted-then-risk-ordered pass, whichever is longer).
  2. Flashing the canary: always permitted (with §11.4.200 verify-after-write identity read-back, already mandated).
  3. Flashing any NON-canary device with artifact fingerprint F: **REFUSED unless a burn-in verdict file for F on the canary exists and is GREEN** — verdict computed from captured §11.4.128 telemetry (crash/ANR census, boot-loop detector, audio XRUN census, Arvus sink probes) plus a **recorded-input-trace replay** (§6.2: replay a §11.4.128-recorded real-user input sequence on the new build, diff its health census against the same trace on the previous build — the honest firmware analogue of shadow traffic). Refusal prints the resolved evidence (§11.4.201(5)).
  4. Escape: none automated; an operator override is an explicit logged decision (§11.4.66), never a flag default.
- **RED-first test:** with no verdict file present, a non-canary flash attempt through the wrapper MUST refuse (exit ≠ 0, naming the missing verdict); with a green verdict for the SAME fingerprint it proceeds; with a green verdict for a DIFFERENT fingerprint it refuses (the §11.4.115(F) fingerprint discipline). Negative control: canary flash never refused by this gate.

- **Does NOT catch:** defects that manifest only after the burn-in window or only on the non-canary device's peripherals (2-device fleet = 1-device canary diversity — honest limit); A/B-style instant rollback (rejected for this target, §6.2 — recovery remains re-flash of the previous image, MASKROM path documented).

8 # 2 # 1

. Accuracy accounting — material vs material

Named-artifact tally (11 checkable artifacts): **REAL: 9** (stryker-js, `thresholds.break`, mutmut, chromadb, sentence-transformers/all-MiniLM-L6-v2, langchain, Istio mirror/mirrorPercentage, `.claude/agents/`, archgate-as-name, CodeHacker-as-name — counting the last two at half-credit each for mischaracterization ⇒ effectively 9/11 ≈ 82% existence). **FABRICATED/ UNLOCATABLE: 2** (`$CLAUDE_TOOL_INPUT` hook environment — proven fabricated with captured docs evidence; `adversarial_runner.py` / `fix_mutants.sh` / `zero-bluff-gates` are acknowledged sketches/placeholders, not counted).

Load-bearing mechanism tally (7 claims an adopter would build on): **SOUND: ~2–3** (mutation testing kills weak tests — true in the diff-scoped form; fuzz/chaos-class adversarial pressure finds real defects; staged canary rollout is real embedded practice). **UNSOUND: 4–5** (the hook wiring that would make GC-1 fire — silently never blocks; 100% whole-corpus kill — unachievable + gaming-inducing; cosine-similarity blocking gate — structural FP engine with self-inconsistent thresholds; ">90% reduction" + "literally cannot produce a false green light" — uncited, and the latter is falsified by our own convergent research (equivalent mutants, oracle weakness, coverage blind complement)).

Net: ≈82% artifact existence (better than material #1's ~2/3), ≈40% mechanism soundness (same failure signature as material #1). The consistent shape across both materials: *real nouns, fabricated verbs* — the tools exist, the integration/ guarantee layer is confabulated, and the confabulated layer is exactly the §11.4.201 false-negative surface (guards that silently never fire, thresholds that force gaming,

guarantees that cannot hold). Material #2 is safe to mine for tool names and unsafe to follow for wiring — verbatim adoption would have installed one more silent no-op guard (GC-1's hook) and two trust-burning gates (GC-3, GC-4).

. Seam-binding scorecard (the programme's convergent question)

Mechanism	Binds where §11.4.85/§1.1 currently prescribe?	How
D1 diff-scoped mutation	**YES**	CM-class gate verdict + Stop-seam consumption; complements (never replaces) hand-authored §1.1 mutations
D2 Go fuzzing	**YES — strongest**	corpus replay executes in every plain `go test` with zero orchestration; crashers become permanent guards mechanically
D2b saboteur role	**Only via its verdict file**	consumed by the Stop-seam completion gate (the fablize-referenced design, TOOLING_STACK_VERIFICATION §6) — as a bare agent role it is prose
D3 arch gate	**YES**	deterministic exit-code gate, always-on tier
D4 size annotator	**NO (by design)**	advisory only — a blocking form would be a §11.4.201(1) false-refusal machine
D5 canary promotion	**YES**	refusal at the flash wrapper — the previously-unbound §11.4.130/§11.4.132 ordering becomes mechanical

The two genuinely unoccupied seams identified by BG-TOOLING-VERIFY (PostToolUse detective hooks; Stop-seam completion gate) are **confirmed, not displaced**, by this material: nothing in GC-1..5 occupies either seam; D1/D2/D2b are *producers* whose verdicts the Stop-seam gate should consume.

. Sources verified (- -)

- StrykerJS thresholds/ `break` : https://stryker-mutator.io/docs/stryker-js/configuration/
- mutmut v3.3.1: https://mutmut.readthedocs.io/en/latest/ · https://github.com/boxed/mutmut
- gremlins v0.6.0 (2025-12-06), 374★, limitations: https://github.com/go-gremlins/gremlins · https://gremlins.dev/
- go-mutesting dormancy: https://github.com/avito-tech/go-mutesting · https://github.com/zimmski/go-mutesting/issues/100
- go-arch-lint: https://github.com/fe3dback/go-arch-lint
- archgate (deterministic ADR rules, v0.50.0 2026-07-17): https://github.com/archgate/cli · https://archgate.dev/
- ArchUnitNET (pattern lineage): https://github.com/TNG/ArchUnitNET
- CodeHacker (competitive-programming scope): https://arxiv.org/abs/2602.20213
- Go native fuzzing (coverage-guided, Go 1.18+): https://go.dev/doc/security/fuzz/ · https://go.dev/blog/fuzz-beta
- Istio mirroring (`mirror` / `mirrorPercentage`): https://istio.io/latest/docs/tasks/traffic-management/mirroring/
- SmartBear/Cisco review study (200–400 LOC): https://static1.smartbear.co/support/media/resources/cc/book/code-review-cisco-case-study.pdf · https://smartbear.com/learn/code-review/best-practices-for-peer-code-review/
- Google small-CL guidance (100/1000 lines + carve-outs): https://google.github.io/eng-practices/review/developer/small-cls.html
- Purushothaman & Perry, TSE 2005 (~4% one-line-change error rate): https://www.researchgate.net/publication/3188494

- Embedded OTA canary/staged/A-B practice: https://memfault.com/blog/ota-testing-101-the-ultimate-guide/ · https://memfault.com/blog/ota-update-checklist-for-embedded-devices/
- chromadb + sentence-transformers/all-MiniLM-L6-v2: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (+ Chroma integration docs)
- Prior captured evidence (not re-fetched): `TOOLING_STACK_VERIFICATION.md` ($CLAUDE_TOOL_INPUT fabrication; the two unoccupied seams; fablize/TruthGuard verification) · `EXTERNAL_RESEARCH.md` R1.3 (Petrović et al., ICSE'21 + TSE'22), R2.5 (OOPSLA'25 PBT ~50×), R4 (SQLite 22 AFL crashers at 100% MC/DC), R5.2 (Google <10% effective-FP bar, CACM 2018), R6.3 (90% override rates).